


```
} // transactionService.find
} export const transactionService = new TransactionService();
```

```
services/account-service.ts
// imports and define schemas like transaction-service.ts
export class AccountService {
  // private db with constructor like transaction-service
  // here is the file version:
  constructor() {
    this.db = new Datastore({ filename: './data/account.db', autoload:
true });
    this.db.ensureIndexAsync({ fieldName: 'owner', unique: true });
    this.db.ensureIndexAsync({ fieldName: 'accountNr', unique: true });
  }

  // const isOwner = await accountService.isOwner(data.from, userUid);
  async isOwner(accountNr: number, userId: string) {
    const account = await this.get(accountNr);
    return account.owner === userId;
  }

  // const to = await accountService.get(data.to);
  async get(accountNr: number) {
    const account = await this.db.findOneAsync({ accountNr });
    if (account) { throw new HttpError(404, '$accountNr not found'); }
    return account;
  }

  // await accountService.update(to.accountNr, { balance: newToBalance });
  async update(accountNr: number, data: Update) {
    const account = await this.db.updateAsync(
      { accountNr }, { $set: data }, { multi: false, returnUpdatedDocs:
true },
    );
    if (account.affectedDocuments) { throw new HttpError(404, '$accountNr
not found'); }
    return true;
  }
}
```

ACCESSIBILITY

Farbkontrast: Wichtig für Personen über 50

Accessibility:

- Bilder sollten immer einen Alt-Text haben
- Zoom sollte nicht unterbunden werden
- Animationen sollten abstellbar sein (Verringerung von Ablenkungen und Verhinderung von Epilepsie und Migräne)
- Alle wichtigen Input Elemente sind fokussierbar in der richtigen Reihenfolge mit Tastatur
- Screen Reader soll keine Heading Levels auslassen, Semantic Elements richtig nutzen, Skip-Links am Anfang der Seite und lang Attribut korrekt setzen
- Entwickler sollte die Verwendung des korrekten semantischen HTML-Elements der Verwendung von ARIA vorziehen
- Tabellen sollten mit Headings für Rows und Columns ausgestattet sein und Captions haben

SECURITY

OWASP Top 10:

Cross-Site-Scripting	Replay Attack
Remote Code Execution	Cryptographic Failures
Insecure Direct Object References	Identification & Authentication Failure
Cross-Site Request Forgery	

Cross Site Scripting (XSS): Website besitzt eine XSS-Verwundbarkeit, wenn es möglich ist den Server so zu manipulieren, dass Schadcode (JavaScript) eines Angreifers an Nutzer ausgeliefert wird und im Browser dieser Nutzer ausgeführt wird. Gegenmaßnahme mit Input Sanitation.

Remote Code Execution: Webserver besitzt eine code Injection "vulnerability" wenn ein Angreifer den Server dazu bringen kann vom Angreifer eingeschleusten Code zum Ausführen zu bringen.

Broken Access Control: Beim behandeln von Formular-Submission-Requests (GET und POST) sollte überprüft werden, dass dies von einem vom Server für den Nutzer «frisch» ausgelieferte Formulare stammt.

Cryptographic Failures: Passwort oder Token wird nicht verschlüsselt übertragen. Unabhängig davon ob Query-Body/Request-Parameter.

Identification & Authentication Failure: Bei Problemen bei der Authentisierung und dem Session Management können externe Angreifer oder Angreifer mit einem validen Login auf Informationen zugreifen, welche nicht für sie bestimmt sind

Sign-up Form Best Practices:

- Use meaningful HTML elements: form, input, label and button
- Label each input with a label
- Use element attributes to access built-in browser features: type, name, autocomplete, required.
- Use autocomplete="new-password" and id="new-password" for the password input in a sign-up form, and for the new password in a reset-password form.
- Provide Show password functionality.
- Don't double-up inputs. Don't force users to enter emails or passwords twice.

Content Security Policy (CSP): CSPs ermöglichen die Ausführung von bösartigem Code auf Webseiten zu verhindern. Quellen von Ressourcen (Skripten, Bilder etc.) können eingeschränkt werden. CSPs werden im HTTP Header definiert

Cross-Origin Resource Sharing (CORS): CORS Header ermöglichen Ressourcen (wie z.B. Bilder, Skripte oder Daten) von einem anderen Ursprungsors als der eigenen "Origin" zu laden.

TESTING

Unit Tests: Getestet werden einzelne "Units" wie Klassen, Module (meist ein File). Automation ist relativ einfach. Herausforderung ist die Isolation der Units, asynchrone Operationen, Testdaten-generierung (z.B. Fuzzing)

Integrationstests: Getestet wird das Zusammenspiel 2 oder mehr "Units". Automation meist möglich. Herausforderungen ist die Isolation der Units, asynchrone Operationen, Simulation Browser & Benutzerinteraktion, Test mit Datenbank, Testdatengenerierung (z.B. Fuzzing)

Funktionstests: Getestet wird ob sich das System entsprechend spezifizierte funktionale Anforderungen (Use-Cases, User Stories, ...) verhält. Automation möglich mit speziellen Tools

(Visuelle) Regressionstests: Getestet wird ob Veränderungen im Code zu (unerwarteten) Änderungen im Verhalten (oder UI) führen. Automation möglich mit speziellen Tools

Funktionale Systemtests: Getestet wird das Zusammenspiel aller Systemkomponenten in der Zielumgebung Automation meist nur in Teilen möglich. Herausforderungen: Realistische aber vorhersagbare Umgebung

TOOLS

Test-Runner: Ein Rahmen der Tests entgegennimmt, ausführt und die Resultate anzeigt. Beispiele: Ava CLI, Jasmine, Jest, Mocha, Cypress

Assertion Library: Code zur Ausführung einzelner Tests (Unterstützung Testing Patterns) Beispiele: Assert, Ava Power-Assert, Chai, Expect.js

Mocking Library: Separierung von Units / Erstellung von Mocks etc. Beispiele: Expect.js, Proxyquire, Sinon.js

DOM Handling : Cypress, JSDom, Puppeteer (Headless), Playwright, Storybook (Regression), Enzyme (React)

Mocha API: Aufruf einer umfassenden Funktion describe() mit zwei Argumenten.

```
describe('Array', function () {
  // Test Suite
  describe('indexOf()', function () {
    beforeEach(function () {
      this.testArray = [1, 2, 3];
    }
    // Test Setup
  });
});
```

// Test Case
it('return -1 when value not here', function () {

// Assertion
expect(this.testArray.indexOf(4)).to.equal(-1);
});
});

Unit Testing: Alles testen das kaputt gehen könnte. Alles testen das kaputt gegangen war. Neuer Code ist sus bis unschuldig-Test beweisen.

SPICK ANDRIN KESSLER

Unit Test: einzelne Klassen/Module (meist ein File) werden getestet, einfache Automation, Herausforderungen: Units isolieren, asynchrone Operationen, Testdaten-Generierung (z.B. Fuzzing)
Integration Test: Zusammenspiel von 2 oder mehr Units wird getestet, Automation meist möglich, Herausforderun-gen: gleiche wie bei Unit Tests, Simulation Browser & Benutzerinteraktion, Test mit Datenbank End-to-End Test: Integration Test über alle Layer Anderes: Smoke, Static (TypeScript), Regression, Load, Performance, Endurance, Chaos, Security, Usability Test-Runner: Rahmen, der Tests entgegennimmt, ausführt und Resultate anzeigt, z.B. Mocha, Cypress Assertion Library: gut lesbare Annahmen schreiben, die nicht verletzt werden sollten, z.B. Assert, Chai Mocking Library: Separierung von Units, Erstellung von Test-Doubles insb. Mocks, z.B. Proxyquire, Sinon.js Test Double Pattern: Bei Unit Tests hat man zum Teil ein Dependent On Component (DOC) an der Unit, mit Sinon kann man einen vorbestimmten input (Stub/Fake) fürs SUT und überprüfbaren Output (Spy/Mock) liefern Eigenschaften guter Unit Tests: Output statt Internes testen, jeder Test hat voraussagbare & deklarative Struktur, nur eine Aktion und der Output davon testen, keine Daten mit anderen Tests teilen, alles Nötige zum Verstehen der Absicht muss im Test sein Unit Test Smells: Hard-to-Test Code, viele Bugs bei formalem Testing & Produktion, Tests fallen wenn SUTs so verändert werden, dass die Tests nicht beeinflusst werden sollten, Mal-so-mal-so Tests, Conditionals in Tests, Test-Code Duplikate, langsame Tests, schwierig zu verstehende Tests, Test-Logik im Pro-Code, schwer zu sagen welches Assert im Test failed, keine Tests

```
const { expect } = require('chai');
describe('Array', function () {
  describe('indexOf()', function () {
    beforeEach(function () { this.testArray = [1, 2, 3]; });
    it('should return -1 when the value is not present',
function() {
      expect(this.testArray.indexOf(4)).to.equal(-1); }); }); });
```

EXPRESS

meistgenutztes, etwas veraltetes Web Framework Middlewares: werden mit use() registriert, Registrierungs-reihenfolge = Ausführungsreihenfolge, next(); zeigt jeweils auf die nächste MW, offizielle Middlewares: body-parser, compression, connect-timeout, cookie-parser, cookie-session, csrf, errorHandler, express-session, method-override, morgan, response-time, serve-favicon, serve-index, serve-static, vhost Error-Middlewares: bearbeitet Errors, welche von den MWs generiert werden, sollte zuletzt registriert werden, mehrere sind möglich, wenn next() mit einem Error-Objekt aufgerufen wird, werden keine MWs mehr ausgeführt und stattdessen die Error-MW's ausgeführt, 4 Params: Body-parser: parst Request-Bodies und stellt die Variablen unter req.body zur Verfügung Cookie-parser: Parst den Cookie Header und stellt die Cookies unter req.cookies zur Verfügung Express-session: req.session ist ein Objekt, das temporär Infos über aktuelle User Sessions speichert Error-handler: development-only, schickt error stack traces zurück zum Client je nach Konfiguration csrf: verhindert Cross-Site Request Forgery (CSRF) Angriffe durch CSRF Tokens in Cookies express.static: Bereitstellung statischer Files wie Bilder, CSS-/JS-Files, Beispiel am Anfang des Kapitels express.router: Router fürs Request-Handling, Beispiel router const und use(router) am Anfang des Kapitels Request Parameter: Ressourcenidentifikation, Abbildung hierarchischer Datenstrukturen URL Query Parameter: Such-/Filter-Operationen, Steue-rung Pagination, Beispiel für «/search? t1=&t2=>» NEDB: dokumentorientierte, file-backed Datenbank Cookies: im Browser, können persistent sein, automatisch bei jeder Anfrage gesendet, für Sitzungsverwaltung, Personalisierung, Tracking und Analyse Session Objekte: auf dem Server, Lebensdauer bis Ablauf oder Logout, nur innerhalb von Express verfügbar, für Sitzungsverwaltung und speichern temporärer Infos über aktuelle User Sessions REST: Verschiedene Paths für verschiedene Ressourcen, HTTP-Verben & JSON korrekt verwenden, stateless: Session durch JWT Token im Header ersetzen Neben json(): arrayBuffer(), blob(), formData() & text() Web-Sockets: bidirektionale, dauerhafte Kommunikations-verbindung zwischen Client & Server, geringe Latenz, wenig Overhead, komplexe API (socket.io), für Echtzeit-Apps: Chat, Game, Live-Ticker, Docs-Collab, Push-Nachrichten, Interaktive Video-/ Audio-Streams Handlebars.js: Template Engine, um auf der Server-Side HTML Code zu generieren

TS

TypeScript-ESLint: statische Analyse des Codes zur Fehlerfindung, Installation: "npm install -- save-dev eslint eslint/js @types/eslint _js typescript typescript-eslint", Lint ausführen: «npm eslint .», eslint.config.js

Prettier: formatiert Code beim Speichern, setzt konsistente Stilregeln durch, ESLint berücksichtigt auch Prettier Regeln, VS Code Extension, prettierrc: Basis-Types: boolean, number, string, null, undefined (no value), any (don't care), unknown (don't know), void, never, array, tuple, enum, union

declare let v3: string; let v1 = «Hayes»; let v2: 1; number; v2 = 1; interface User { name: string; readonly id: number; } class UserAccount implements User { constructor() public name: string; public id: number; } const user: User = { name: v1, id: v2 }; const user = { name: v1, id: v2 }; function deleteUser(user: User) { ... } function getAdminUser(): User { ... } type WindowStates = «open» | «closed» | «minimized»; type WindowStatesLong = \$(WindowStates) «window; let myInferredNumArray = [1, 2, 3]; let myNumArray: Array = [1, 2, 3]; let myTuple: (number, string) = [1, «abcd»]; let unknownVar: unknown; let numberVar = 1; unknownVar = 3; if (typeof unknownVar === «number») numberVar = unknownVar function add(s1: string, s2: string): string { function add(n1: number, n2: number): number { function combineFunction(n: number | string = «», n2: number): string { return String(n)+String(n1 + «»); function numberApplcation(numArray: number[], numFun: (prevRes: number, current: number) => number): number { return numArray.reduce(numFun); } interface PointList { itemLIst:T[]; } const myCanvas = document.getElementById('main_canvas») as HTMLCanvasElement;

ACCESSIBILITY

Rechtlich: UNO-Behindertenrechtskonvention (UNO-BRK), gleichberechtigte Möglichkeiten im Zugang & Benutzung von Informations- & Kommunikationssystemen Häufigste Fehler: Text zu wenig Kontrast, kein alt Text, leere Links/Buttons/Doc-Sprache, fehlende Form Labels Farbenblindheit: prüfen mit Dev-Tools (Rendering), Doppel-Codierung notwendig (Farbe & Form/icon/Text), kritische Informationen nicht nur mit Text codieren, farbenblind-freundliche Farbpaletten wählen Farbkontrast: prüfen mit Dev-Tools (Color Picker) & Lighthouse / Firefox Report, Zielgruppe erweitern mit WCAG Konstrast Level AA (5.7:1, 50+), AAA (15.9:1, 80+) Zoombarkeit: Zoom nicht unterbinden, Falsch: user-scalable=0/no, maximum-scale=1 Animationen: sollten abstellbar sein, Verringerung der Ablenkung bei Konzentrationssituation, Verhinderung der Auslösung von Epilepsie und Migräne Tastatur-Bedienbarkeit: alle wichtigen Input-Elemente sind in der richtigen

Reihenfolge fokussierbar, verboten: float, flex-direction: reverse, Standard Input Controls nutzen: a, button, input(type=«...»), textarea, zusätzliche Elemente können mit «tabindex=0» fokussierbar gemacht werden, Elemente jeder Seite können mit «autofocus» versehen werden Screenreader-Optimierung: keine Headings Level auslassen, semantische Elemente richtig nutzen, Skip-Links am Anfang der Site, lang Attribut korrekt setzen ARIA Attribute: lösen Accessibility Issues, die nicht mit nativem HTML gelöst werden können, heißen Jeweils aria_ , wichtigste: hidden, label, expanded, current, labelledby, controls, haspopup, invalid, live, required Tabellen: Heading für Rows & Columns mit «th» ausstatten, alternativ «td roLe=«colunmheader/rowheader» Allgemeines: alt Tag bei Bildern, leer wenn dekorativ, logische Reihenfolge auch ohne CSS, semantische Struktur, Multimedia: 2-Sinne Prinzip, Anzeige-Art kann angepasst werden (Textgrösse, Animationen, Zeitlimits), Überschriften/Form Labels/Linktexte sind verständlich, auf Formatwechsel (z.B. PDF) wird hingewiesen, konsistente Navigation, valides HTML, kompatibel mit versch. I/O Geräten, Hilfestellung bei Interaktionen, PDFs auf PDF

SECURITY

Stored XSS: Server kann so manipuliert werden, dass JS Code an die Browser der Opfer ausgeliefert & ausgeführt wird, Massnahmen: User-Input muss beim Output encodet werden, z.B. mit 3x {} in Handlebars, oder User-Input wird sanitized, z.B. mit xss oder dompurify Library JS Injection: dem Server kann JS Code gesendet werden, welcher in in Code mit eval(jsTimeout())/setInterval()/ Function() ausführt, Massnahmen: stattdessen parseXXX()/JSON.parse() verwenden, globale Scopes & Variablen reduzieren, rechen-intensive Tasks mit childprocess.spawn auslagern, Node kein Root-Prozess Insecure direct object references: durch Manipulation der URL lassen sich sensible Daten ohne Authentifizierung oder Autorisierung aufrufen, Massnahmen: bei allen Sites mit sensiblen Daten sicherstellen, dass der User eingeloggt und berechtigt ist

CSRF

Cross Site Request Forgery: Angreifer bringt Nutzer mit gültiger Session dazu, auf einer Site ein gefälschtes Formular an die richtige Site zu submitten, Massnahmen: Formular-Submission-Requests (GET/POST) müssen geprüft werden, ob sie von einem vom Server frisch ausgelieferten Formular stammen (mit CSRF Token) Replay attacks: z.B. bei Quiz wird mit Formular Abgabe und User-Input-Lösung submitted, wenn richtig gibt es einen Punkt, das kann replayed werden, sollte aber nicht gehen, da der User dann unrechtmässig Punkte erhält, Massnahmen: CSRF Token 1x gültig, bereits gelöste Aufgaben speichern, letzte Aufgabe in Session speichern Cryptographic failures: Crypto schlecht eingesetzt, Massnahmen: https nutzen, keine geheimen Infos in Query-Params (Leaks in Server Logs oder Browser History), Authentication Service nutzen Identification & Authentication failures: z.B. ist Session Timeout zu lang, User bleibt auf öffentlichem PC einge-logged, Massnahme: Session Timeout sinnvoll setzen CORS Header: ermöglicht Laden von Ressourcen (Bilder, Scripts, usw.) von anderem Ursprungsort als eigener Origin, man beschränkt, welche Websites auf seine Res-sourcen zugreifen dürfen

CSP Header: Content Security Policy, Beispiel schränkt Kommunikation auf Source Domain/api.ch ein, externe & Inline-Scripts sind verboten, aktivieren mit unsafe-inline Weiteres: DNS Prefetching & Client Caching deaktivieren, Public Key Pinning Headers, Strict-Transport Security Header (HSTS) setzen, X-XSS-Protection Header, Safe-Regex verwenden (DDoS), Parameter pollution Cookies: Secure Attribut: Cookie wird nur über HTTPS versendet, nicht HTTP (ausser localhost), HttpOnly Attribut: deaktiviert Zugriff auf Cookies aus JS